Sorting Algorithms-2

Table of Contents

Radix Sort

Radix Sort

- Radix sort is an algorithm that sorts numbers by processing digits of each number either starting from the least significant digit (LSD) or starting from the most significant digit (MSD).
- The idea of Radix Sort is to do digit by digit sort starting from least significant digit to most significant digit. Radix sort uses counting sort as a subroutine to sort.

Radix Sort

- Algorithm:

- Step1: Take the least significant digit of each element
- Step2 : Sort the list of elements based on that digit
- Step3 : Repeat the sort with each more significant digit

Radix Sort

- Assume the following Array:

170	45	75	90	802	24	2	66

Radix Sort

- Step1: Sorting by least significant digit (1s place)

170	45	75	90	802	24	2	66

170	90	802	2	24	45	75	66

Radix Sort

- Step2: Sorting by next digit (10s place)

170	90	802	2	24	45	75	66

802	2	24	45	66	170	75	90

Radix Sort

- Step3: Sorting by most significant digit (100s place)

802	2	24	45	66	170	75	90

2	24	45	66	75	90	170	802

Radix Sort

- The Sorted list will appear after three steps

170	45	75	90	802	24	2	66
170	90	802	2	24	45	75	66
802	2	24	45	66	170	75	90
2	24	45	66	75	90	170	802

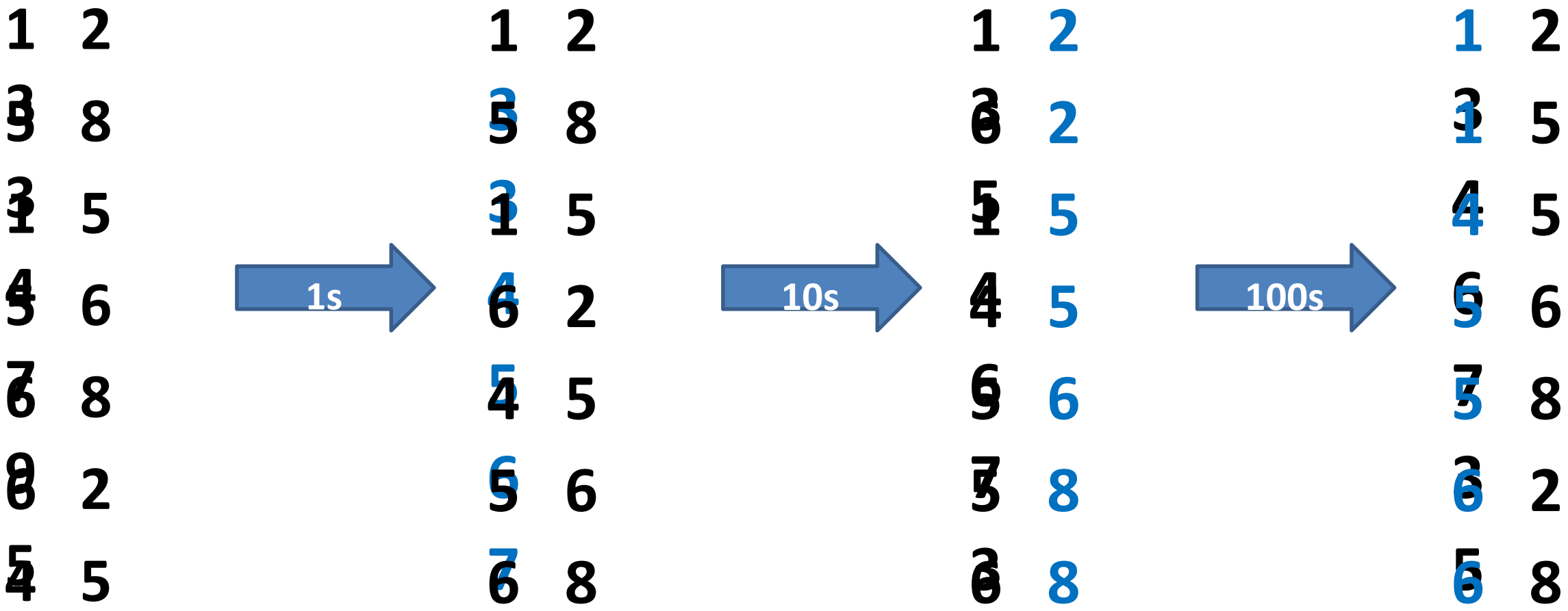
Radix Sort

- Array is now sorted

2	24	45	66	75	90	170	802

Radix Sort

- Example 2



Radix Sort

- C++ Code

Try Yourself

Radix Sort

❑Time Complexity:

Radix Sort is a linear sorting algorithm. Radix Sort's time complexity of **O(nd)**,

where **n** is the size of the array
and **d** is the number of digits in the largest number.

It is not an in-place sorting algorithm because it requires extra space.

Merge Sort

Merge Sort

- Merge Sort is a Divide and Conquer algorithm. It divides input array in two halves, calls itself for the two halves and then merges the two sorted halves.

Algorithm:

- Step1: Divide the list recursively into two halves until it can no more be divided
- Step2 : Merge (Conquer) the smaller lists into new list in sorted order

Merge Sort

- Assume the following Array:

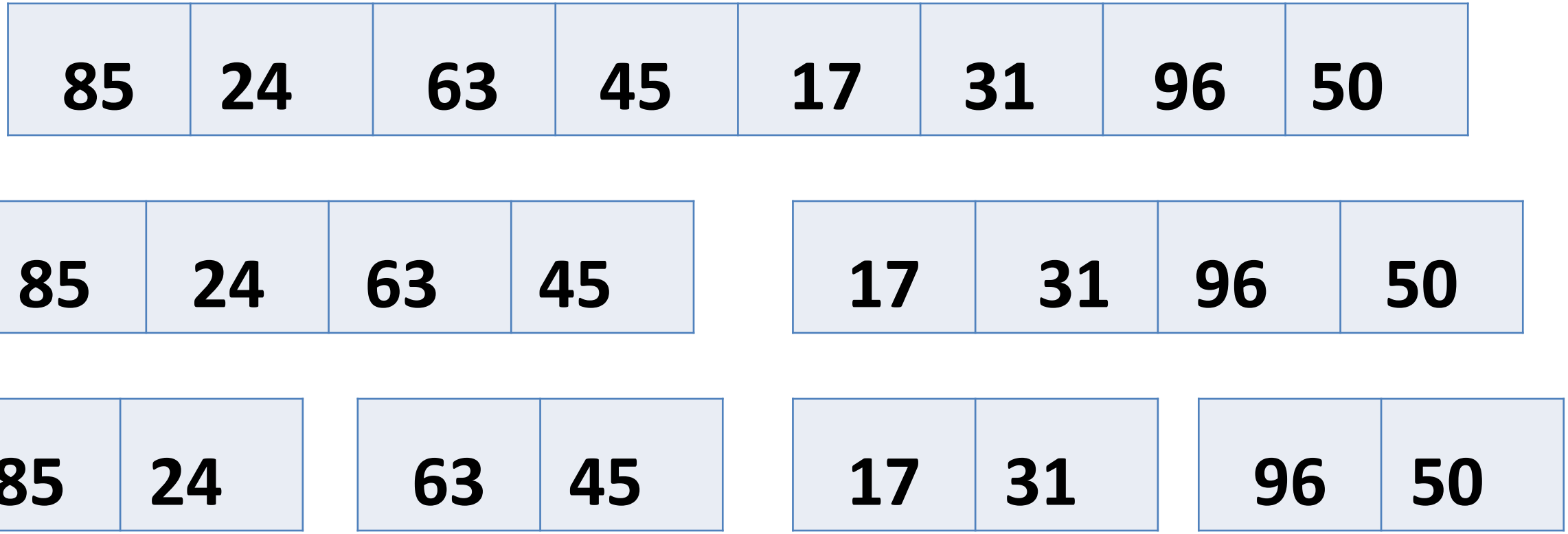
85	24	63	45	17	31	96	50

Merge Sort

- Divide

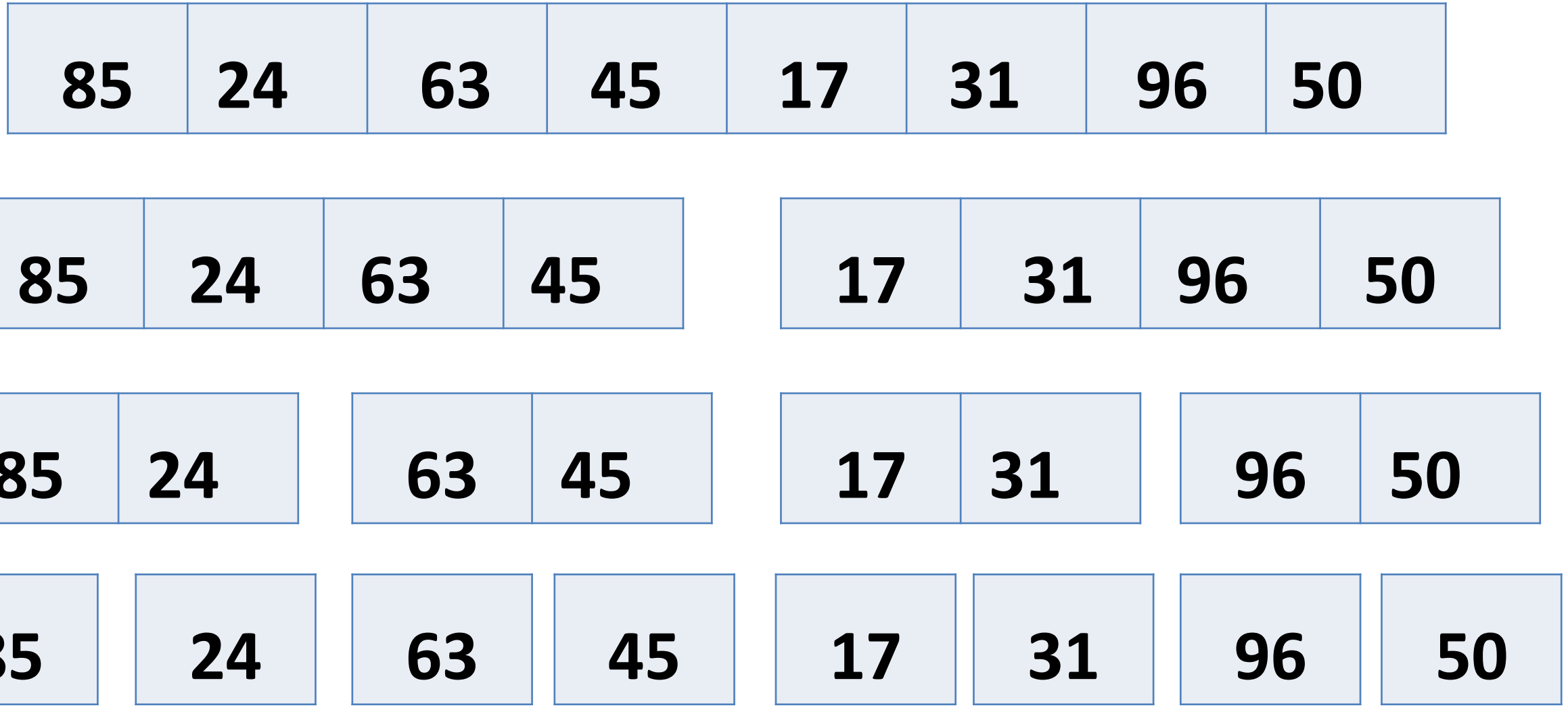
Merge Sort

- Divide



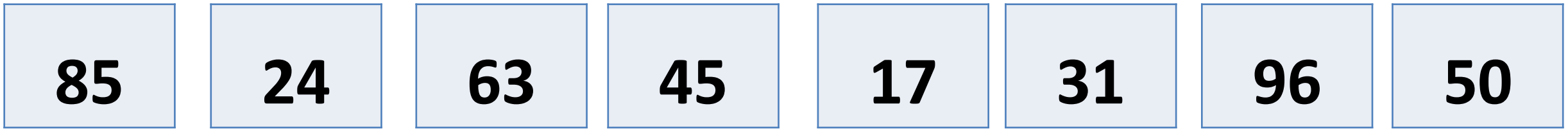
Merge Sort

- Divide



Merge Sort

- Sort & Merge



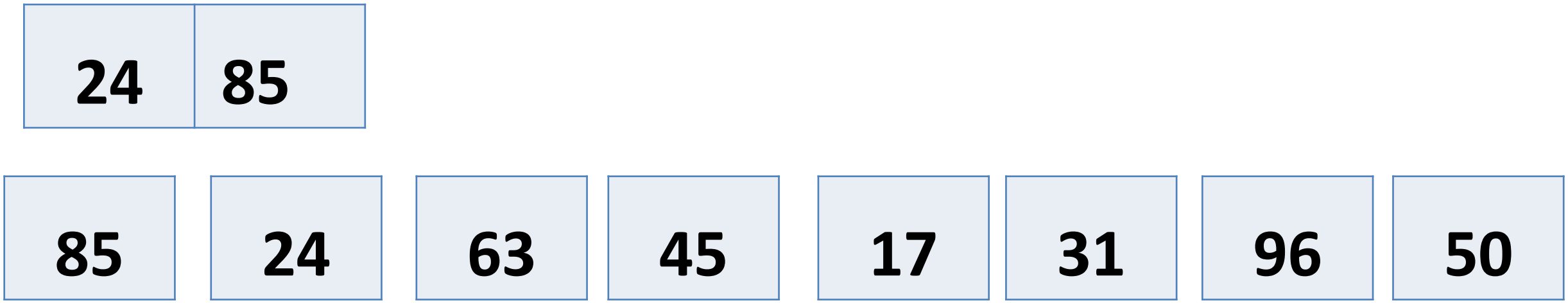
Merge Sort

- Sort & Merge

85	24	63	45	17	31	96	50

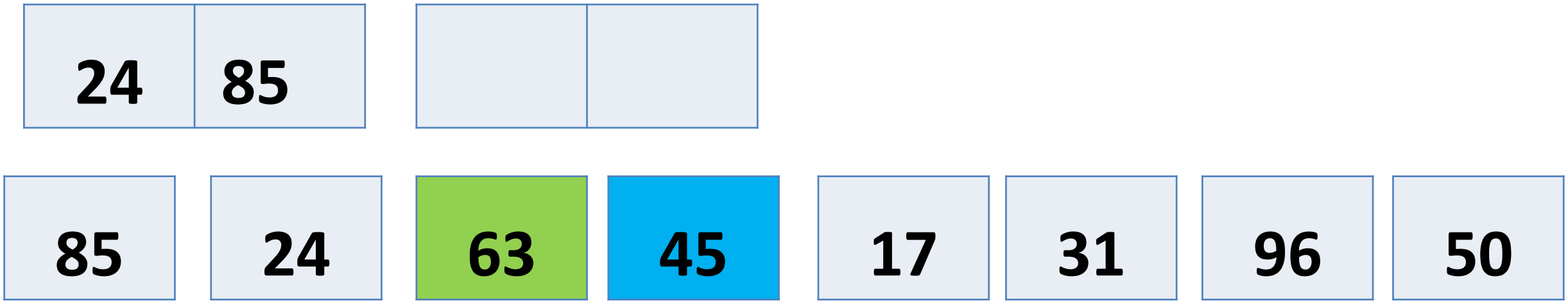
Merge Sort

- Sort & Merge



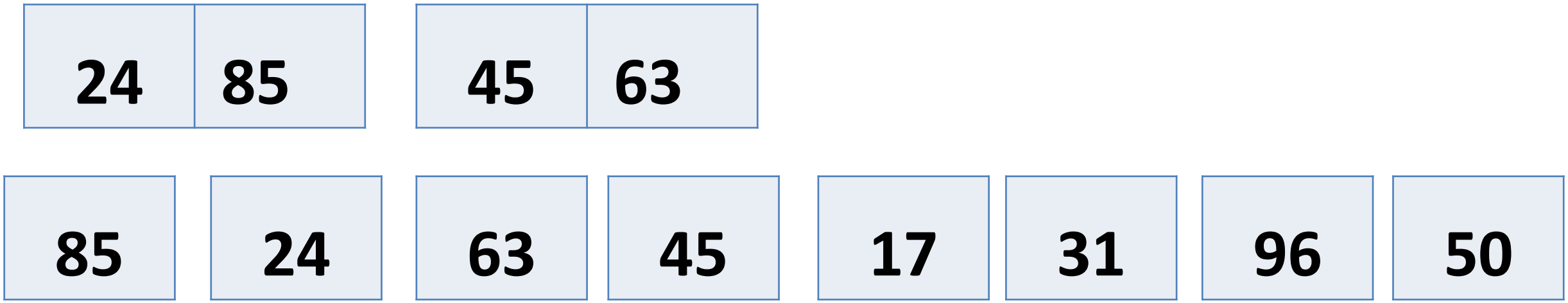
Merge Sort

- Sort & Merge



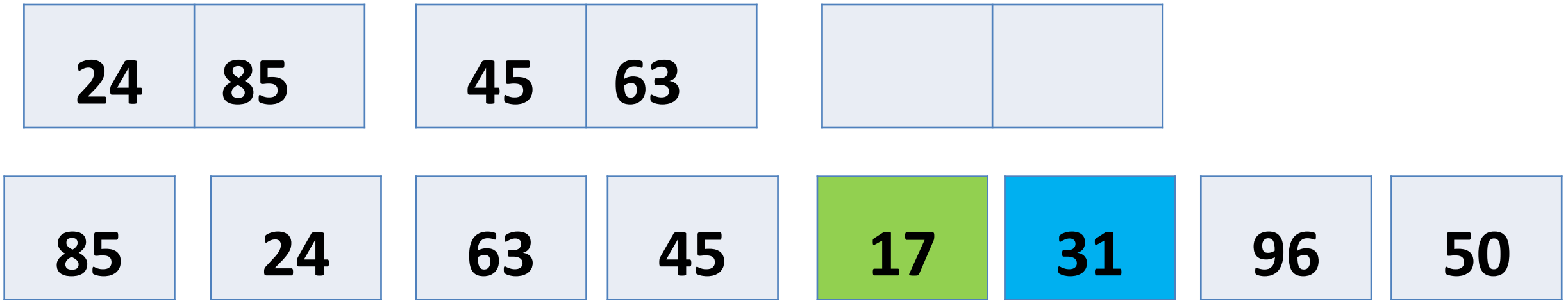
Merge Sort

- Sort & Merge



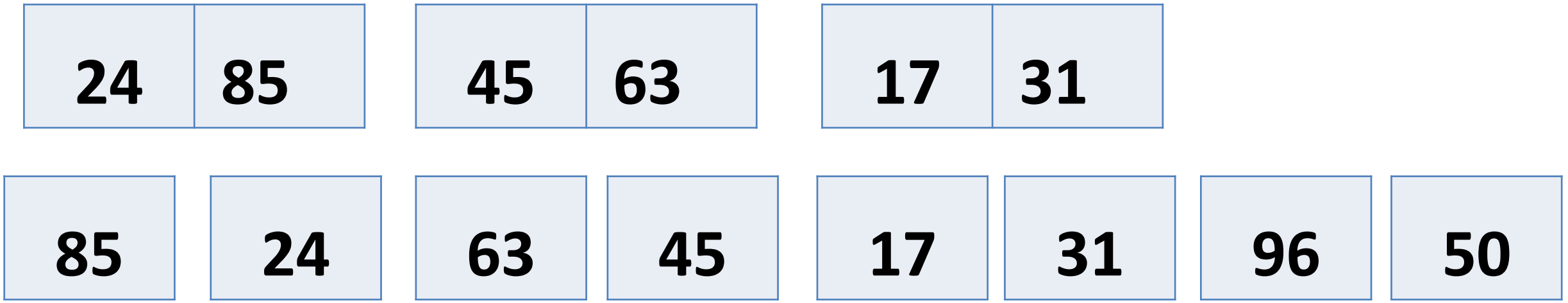
Merge Sort

- Sort & Merge



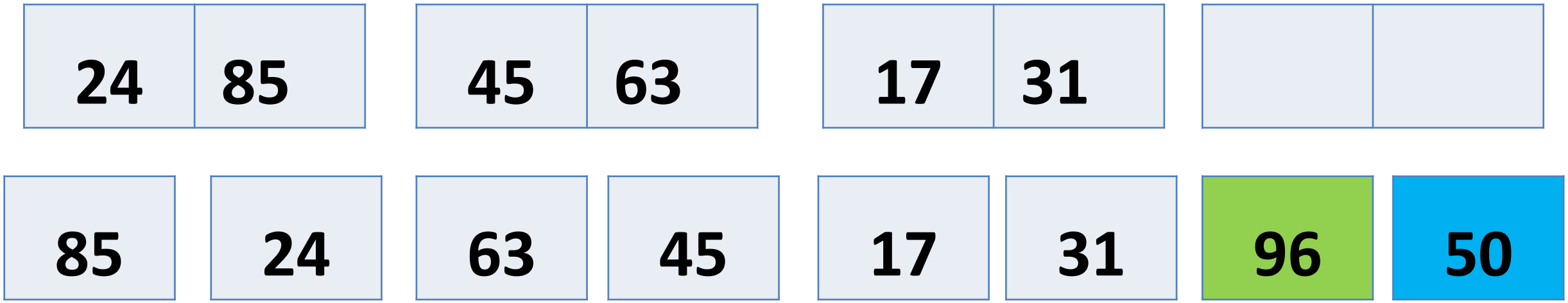
Merge Sort

- Sort & Merge



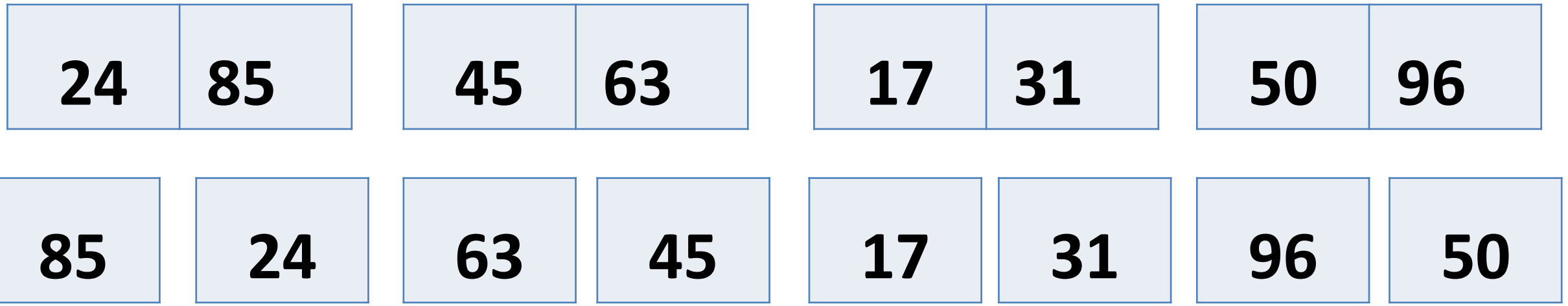
Merge Sort

- Sort & Merge



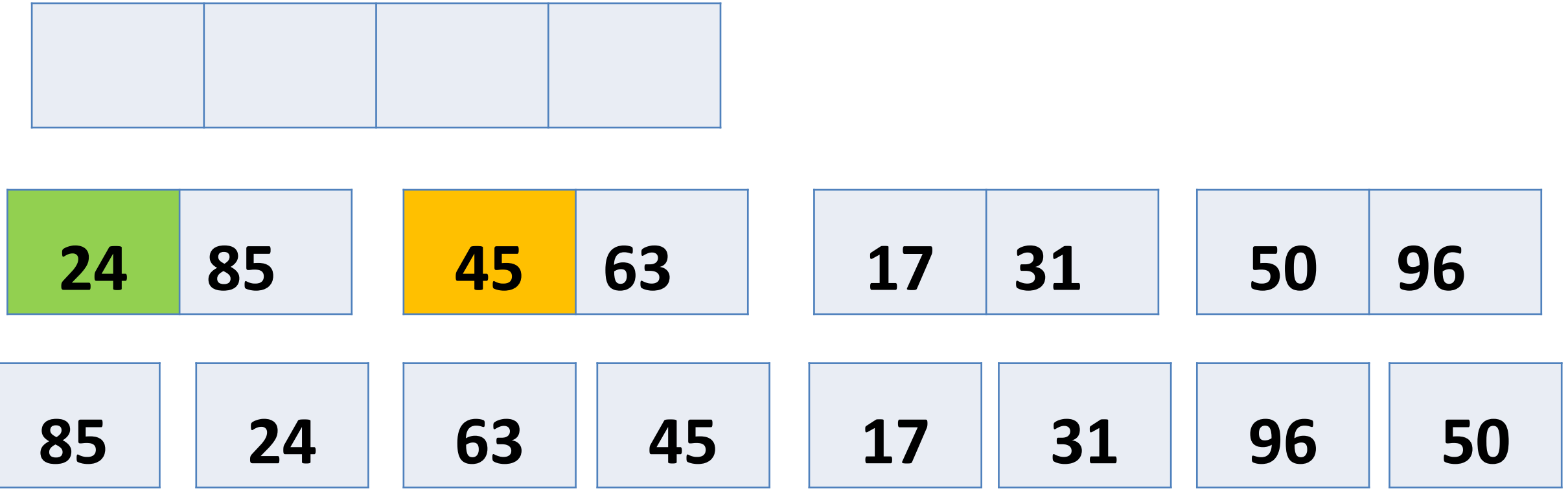
Merge Sort

- Sort & Merge



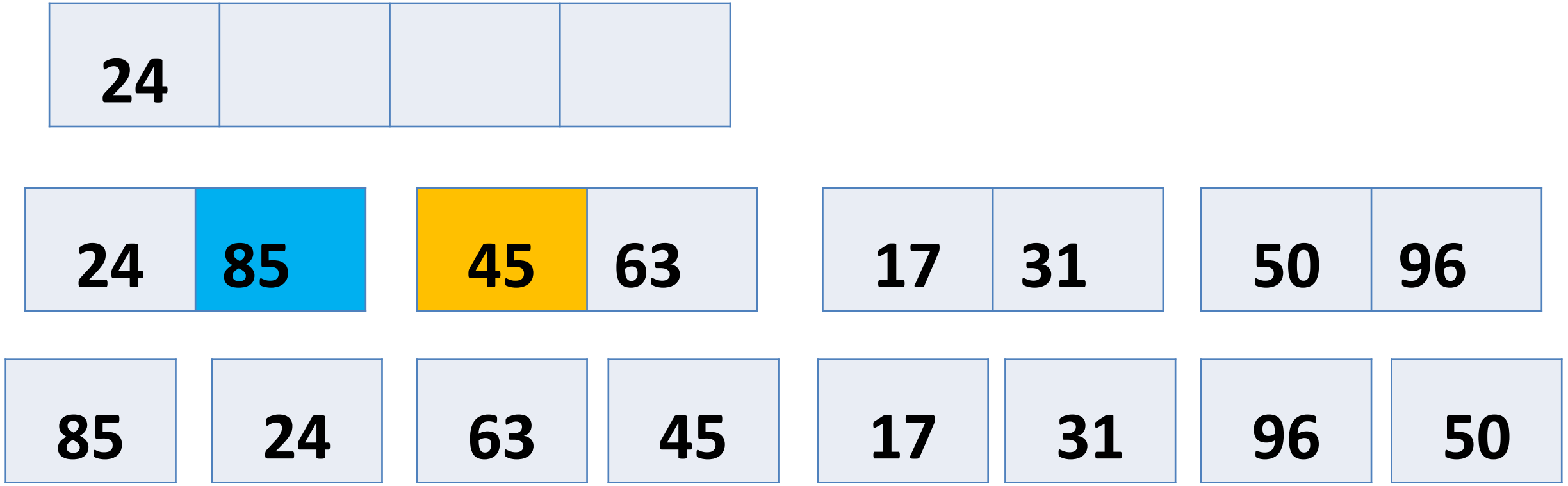
Merge Sort

- Sort & Merge



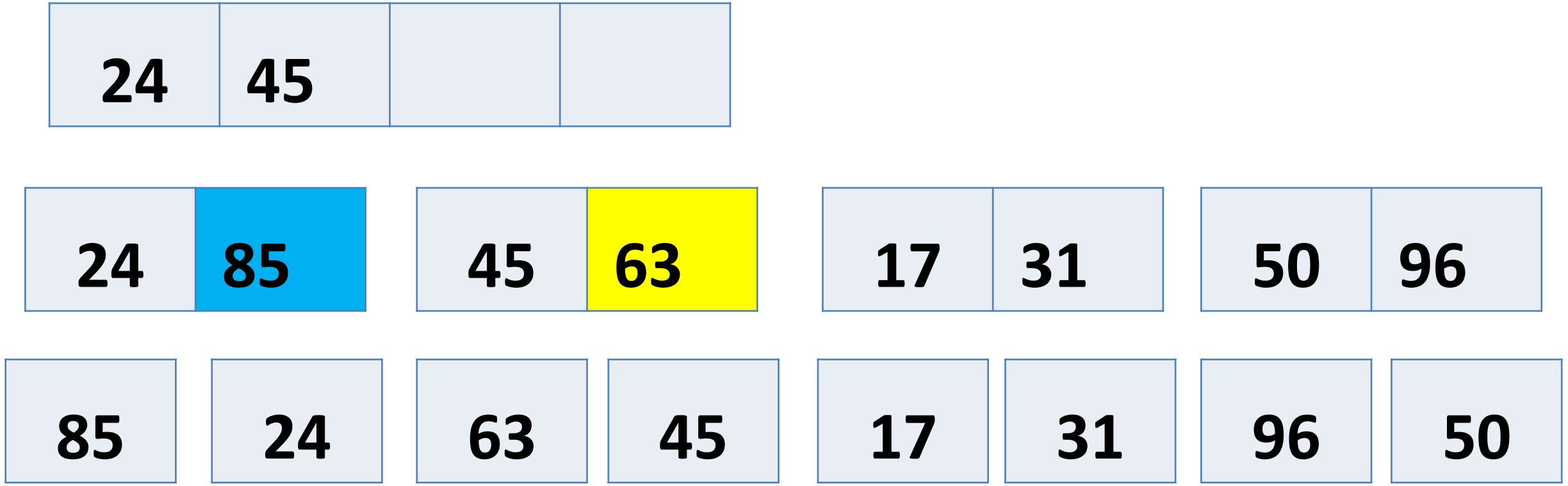
Merge Sort

- Sort & Merge



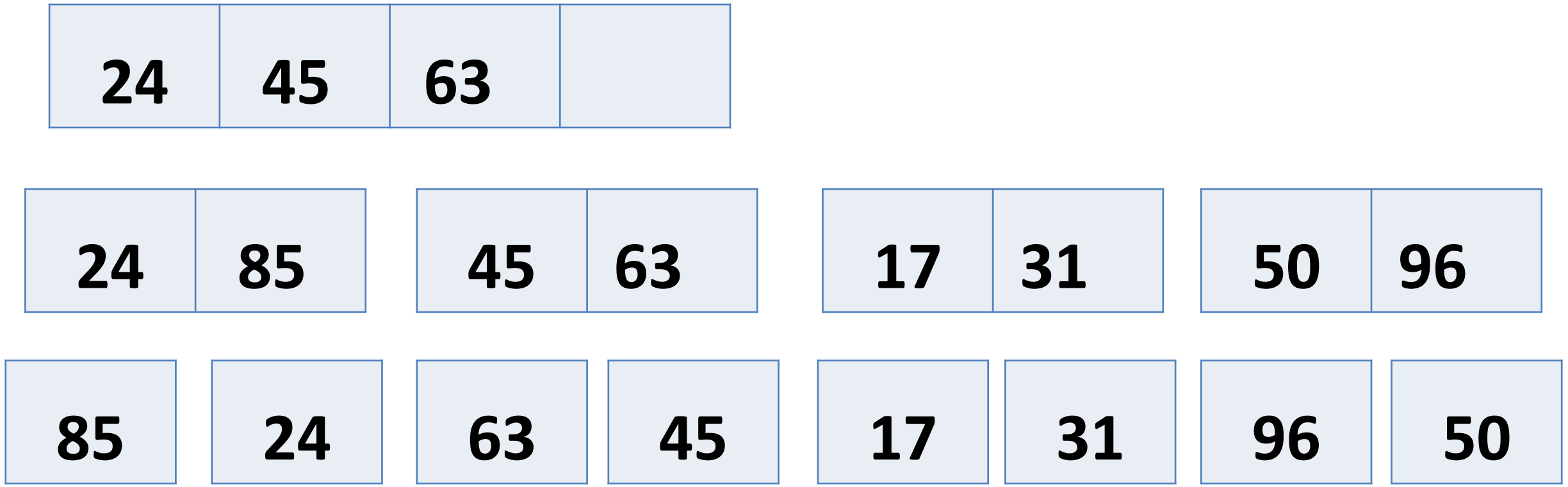
Merge Sort

- Sort & Merge



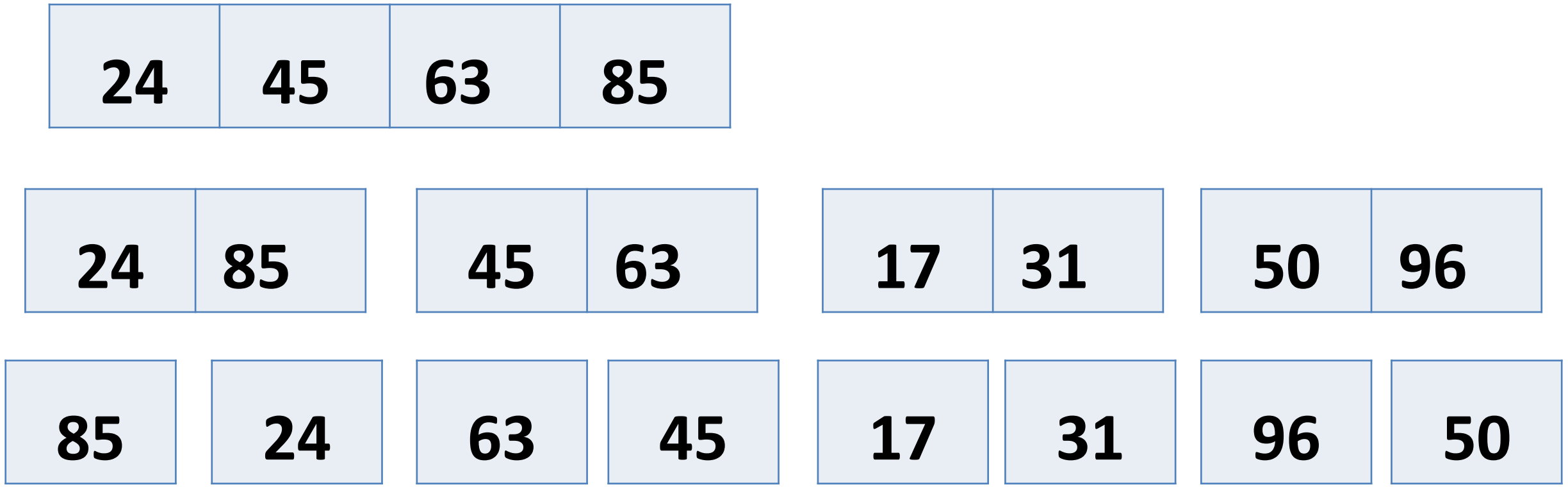
Merge Sort

- Sort & Merge



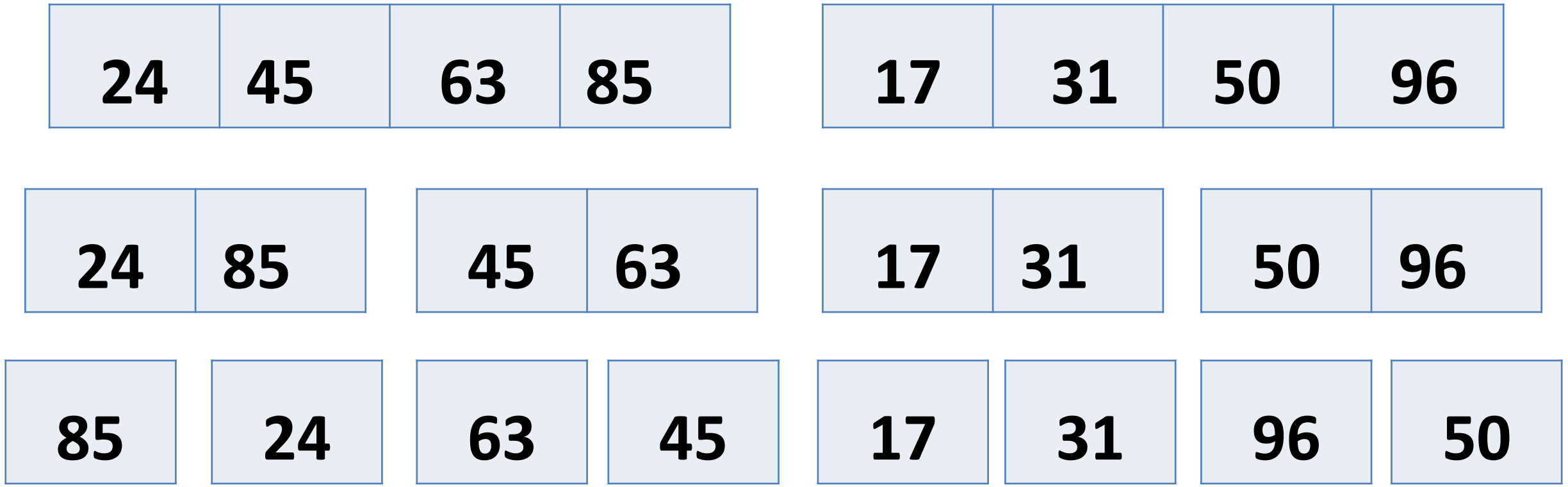
Merge Sort

- Sort & Merge



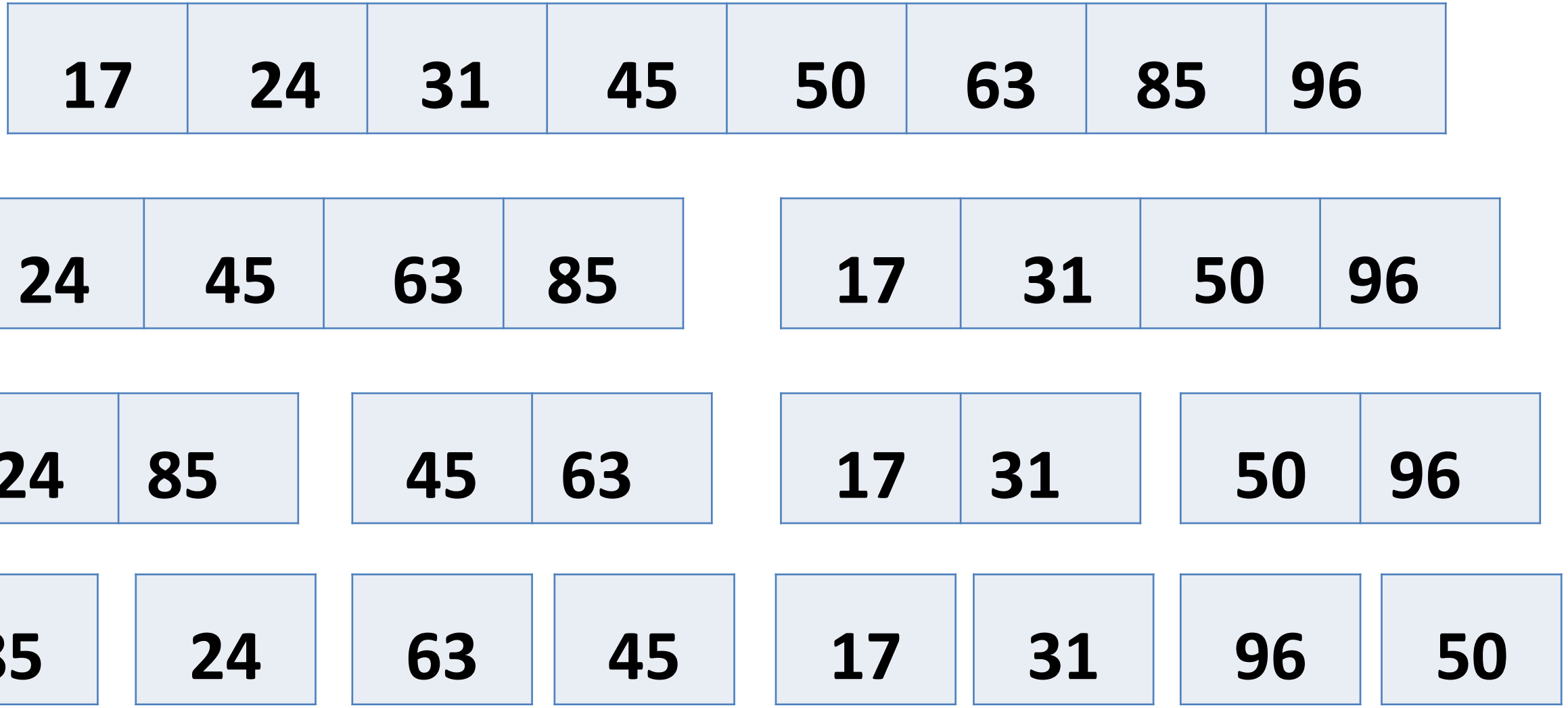
Merge Sort

- Sort & Merge



Merge Sort

- Sort & Merge



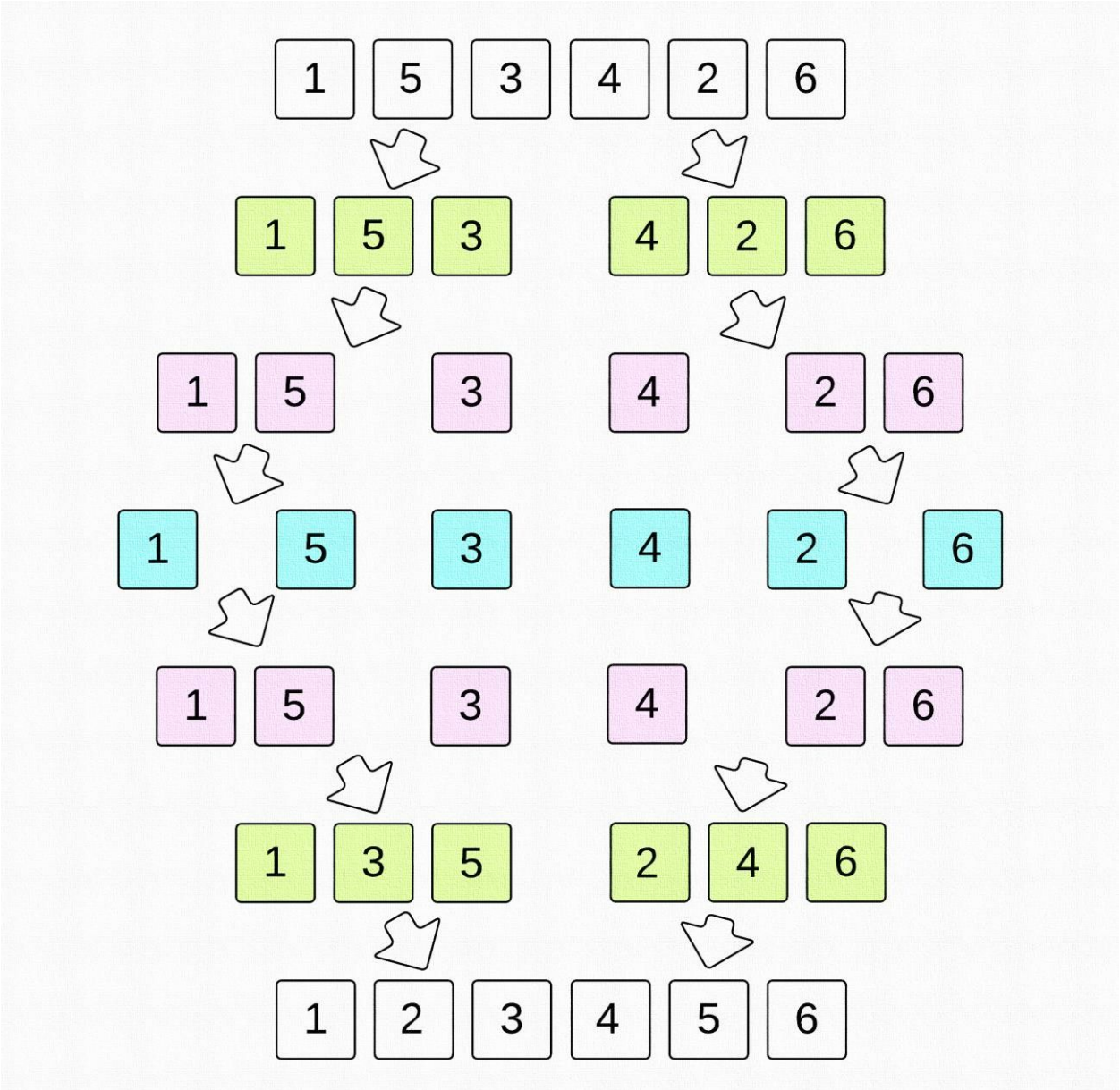
Merge Sort

- Array is now sorted

17	24	31	45	50	63	85	96

Merge Sort

- Example 2



Merge Sort

Try C++ code Yourself, Python Code is added here using recursive (mergeSort) function.

Merge Sort

Python Code

```
def merge(arr, l, m, r):
    n1 = m - l + 1
    n2 = r- m
    L = [0] * (n1)
    R = [0] * (n2)
    for i in range(0 , n1):
        L[i] = arr[l + i]
    for j in range(0 , n2):
        R[j] = arr[m + 1 + j]
    i = 0     # Initial index of first subarray
    j = 0     # Initial index of second subarray
    k = l     # Initial index of merged subarray
    while i < n1 and j < n2 :
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    while i < n1: # Copy the remaining elements of L[]
        arr[k] = L[i]
        i += 1
        k += 1
    while j < n2: # Copy the remaining elements of R[]
        arr[k] = R[j]
        j += 1
        k += 1
```

Merge Sort

```
def mergeSort(arr,l,r):
    if l < r:
        m = int((l+(r-1))/2)
        mergeSort(arr, l, m)
        mergeSort(arr, m+1, r)
        merge(arr, l, m, r)
    return arr
```

Merge Sort

```
arr = [12, 11, 13, 5, 6, 7]
print(mergeSort(arr,0,len(arr)-1))
```

Merge Sort

❑Time Complexity: O(n * log(n))

- The time to merge sort n numbers is equal to the time to do two recursive merge sorts of size n/2 plus the time to merge, which is linear.

- We can express the number of operations involved using the following recurrence relations

  $T(1)=1$

  $T(n)=2T(n/2)+n$

- Going further down using the same logic

  $T(n/2)=2T(n/2)+n/2$

- Continuing in this manner, we can write

  $T(n)=nT(1)+n\log n$

  $=n+n\log n$

  $T(n)=0(n\log n)$

THANK YOU